

AS-BUILT TECHNICAL REPORT

Finance LLM V2 네이티브 검색 마이그레이션

진행 과정, 최종 아키텍처, 전환 절차 및 운영 수명주기

문서 기준일: 2026-07-19

문서 성격: 구현 완료 상태를 기준으로 한 상세 기술 기록

대상 범위: V1 SQLite + LangChain FAISS 저장 구조에서 V2 네이티브 검색 구조로의 전환

제외 범위: 테스트 코드, 테스트 절차, 성능 측정 및 테스트 결과

핵심 결론: 이번 변경은 LangGraph 기반 질문 처리 흐름을 교체한 작업이 아니다. 기존 LangGraph 오케스트레이션은 유지하고, 그 안에서 VectorDB 검색과 데이터 갱신이 사용하는 저장·조회·게시·복구 계층을 V2 네이티브 구조로 교체한 작업이다. V1 데이터는 보존되며, V2가 쓰기 가능한 첫 successor를 게시한 뒤에만 V1 fallback이 영구적으로 닫힌다.

목차

1. 문서 목적과 범위
2. 마이그레이션 배경과 문제 정의
3. 설계 및 구현 진행 과정
4. 최종 역할 분리: LangGraph와 V2 Retrieval
5. 소스 코드 구조와 수명주기 경계
6. 런타임 데이터 배치 구조

7. V2 식별 체계와 카탈로그 데이터 모델
8. V1에서 V2로 전환되는 상세 과정
9. 런타임 선택과 읽기 경로
10. V2 쓰기와 전체 corpus successor 생성
11. 원자적 publication과 내구성 프로토콜
12. 동시성 제어와 잠금 계층
13. 시작 복구, 정리 및 지원 정보
14. 애플리케이션 통합 방식
15. V1 보존 정책과 지원 종료 이후 정리
16. 운영 제약과 핵심 원칙
17. 구현 파일 안내와 용어 정리

1. 문서 목적과 범위

이 문서는 Finance LLM의 검색 저장 구조를 v1에서 v2로 마이그레이션하면서 어떤 문제를 해결했고, 어떤 단계로 구현했으며, 최종적으로 코드와 런타임 데이터가 어떻게 구성되는지를 설명한다. 단순 사용 안내서가 아니라 향후 유지보수자가 설계 의도와 삭제 가능 경계를 다시 추론하지 않아도 되도록 남기는 as-built 기술 기록이다.

설명 대상은 검색 데이터의 식별, 변환, 네이티브 조회, 전체 corpus 갱신, snapshot publication, 장애 복구, 잠금, v1 호환 브리지, 사용자용 원클릭 전환 및 소스 폴더 수명주기다. 앱의 질문 rewrite, routing, RDB 검색, 답변 생성, Streamlit 화면 등 기존 LangGraph 상위 흐름은 변경되지 않은 경계로 다룬다.

구분	포함 내용	문서화 이유
포함	마이그레이션 진행 경과와 주요 결정	왜 현재 구조가 되었는지 설명
포함	v2 조회·쓰기·publication·recovery 아키텍처	운영 중 데이터 일관성과 장애 대응 이해
포함	v1/v2 역할, 보존 범위, 삭제 시점	전환 코드와 영구 코드를 혼동하지 않기 위함
제외	테스트 구현, 테스트 명령, 측정치와 결과	요청 범위에 따라 기술 본문에서 제외
제외	일반 RAG 프롬프트와 UI 세부 기능	이번 저장 구조 마이그레이션의 직접 범위가 아님

문서의 상태 표현은 현재 소스 코드 구조를 기준으로 한다. 특정 사용자의 실제 데이터 크기, 리포트 수, snapshot ID, publication generation 같은 실행별 값은 문서에 고정하지 않는다.

2. 마이그레이션 배경과 문제 정의

2.1 v1 검색 저장 구조

v1은 data/reports.db에 리포트 메타데이터와 parent chunk를 저장하고, data/vector_db/index.faiss에 벡터를 저장하며, data/vector_db/index.pkl에 LangChain FAISS wrapper

의 docstore와 ordinal-to-document 매핑을 저장한다. 검색 결과의 의미는 FAISS 파일과 pickle 파일, SQLite 행이 동시에 같은 상태일 때만 성립한다.

이 구조는 초기 구현에는 단순하지만, 벡터 ID의 논리적 의미가 pickle 내부 매핑에 숨고, 개별 파일의 교체 순서에 따라 서로 다른 시점의 데이터가 섞일 수 있다. 또한 임베딩 모델, 추출기, chunk 정책 같은 의미 공간의 설정이 snapshot과 강하게 결합되어 있지 않아, 시간이 지난 뒤 동일 검색 공간인지 증명하기 어렵다.

2.2 해결해야 했던 핵심 문제

- 논리적 문서·parent-child 정체성을 파일 순서나 프로세스 메모리가 아니라 결정적 해시로 표현해야 했다.
- FAISS에는 벡터와 snapshot-local 정수 ID만 두고, 텍스트·메타데이터·membership의 권위는 SQLite가 가져야 했다.
- 새 index를 만드는 동안 현재 사용 중인 index를 변경하지 않고, 완성된 전체 corpus snapshot만 게시해야 했다.
- publication 도중 프로세스 종료나 재시작이 발생해도 어느 snapshot이 활성인지 파일명 추측 없이 복구해야 했다.
- v2가 안정적으로 쓰기 가능한 상태가 되기 전까지 v1을 보존하고, 검증되지 않은 상태에서는 읽거나 쓰기를 임의로 계속하지 않아야 했다.
- 마이그레이션 한 번을 위한 코드와 앱이 계속 사용하는 코드를 물리적으로 분리해야 했다.

2.3 설계 목표

목표	v2 설계 응답
결정성	동일 입력과 동일 embedding profile이면 동일 report_uid, parent_uid, chunk_uid와 snapshot identity를 생성
원본 보존	V1 reports.db, vector_db, downloaded, conversations.db를 교체하거나 삭제하지 않음
원자적 전환	격리된 retrieval 디렉터리를 완성한 뒤 단일 rename으로 활성화

목표	v2 설계 응답
단일 권위	SQLite retrieval_runtime 행만 active snapshot pointer의 권위로 사용
불변 snapshot	고유 경로에 no-overwrite 방식으로 게시하고 hash-size-dimension-metric-count를 함께 고정
복구 가능성	journal, commit intent, checkpoint, committed floor를 이용해 중단된 publication을 재생 또는 복구
Fail closed	정체성이나 내구성 증거가 모순되면 오래된 v1을 추측해서 열지 않고 안전하게 중단

3. 설계 및 구현 진행 과정

구현은 저장 형식만 바꾸는 단일 단계가 아니라 역할 경계를 먼저 정한 뒤, 영구 검색 엔진, 전환용 변환 계층, 앱 연결부, 운영 안전장치를 순차적으로 분리하는 방식으로 진행되었다. 다음 단계는 코드의 현재 구조가 형성된 논리적 순서를 요약한다.

단계	진행 내용	결과
1	V1과 V2의 역할, LangGraph와 retrieval의 경계를 정의	LangGraph는 유지하고 DB/retrieval 계층만 교체한다는 원칙 확정
2	네이티브 identity, schema, raw FAISS snapshot 계층 구현	pickle 없는 V2 저장 계약과 결정적 ID 체계 확보
3	V1 copied-install 평가, span 복원, seed 변환, compatibility bundle 구현	V1 원본을 쓰지 않고 epoch-zero V2 seed 생성 가능
4	bootstrap, dispatch, native reader, repository를 앱 검색 노드에 연결	한 LangGraph 노드에서 legacy, compatibility, native runtime을 안전하게 선택
5	whole-corpus build, publication, recovery, garbage collection 구현	off-path 후보 생성과 crash-safe snapshot 전환 가능
6	update lock, writer lock, runtime guard, launcher guard 연결	크롤링·임베딩·마이그레이션 간 동시 쓰기 차단
7	영구 코드와 전환 코드를 폴더 단위로 재배치	src/retrieval과 src/migrations/v2의 수명주기 경계 명확화
8	MIGRATE_V2.bat 및 사용자용 전환 흐름 구성	일반 사용자가 별도 JSON 작성 없이 표준 설치를 전환할 수 있는 진입점 제공

3.1 진행 과정에서 확정된 주요 결정

- V2는 V1 index.pkl을 새 포맷으로 계속 사용하는 구조가 아니라, SQLite membership과 raw FAISS snapshot으로 의미를 재구성한다.
- 첫 마이그레이션 seed는 기존 V1 벡터를 재사용하지만, 실제 쓰기 가능한 첫 successor는 전체 PDF corpus를 다시 추출·chunk·embedding하여 만든다.

- 부분 갱신 API를 두지 않는다. 소스 변화는 하나의 완전한 successor snapshot으로 표현한다.
- 영구 retrieval 패키지는 마이그레이션 패키지를 import하지 않는다. 전환 패키지가 영구 retrieval 계약을 사용하는 방향으로 의존성을 둔다.
- epoch zero는 임시 읽기 전용 단계이며, 일반 사용자용 성공 경로는 반드시 write_epoch가 1 이상인 successor까지 완료한다.
- v1 지원 종료는 마이그레이션 성공과 같은 사건이 아니다. 별도의 지원 종료 결정이 있어야 전환 코드와 compatibility bridge를 삭제한다.

결과적으로 현재 구조는 'V1 LangGraph 앱을 V2 앱으로 다시 작성'한 것이 아니라, '기존 앱이 호출하는 검색 포트를 V2 네이티브 data plane으로 교체하고 전환 기간에만 V1 bridge를 둔 구조'다.

4. 최종 역할 분리: LangGraph와 V2 Retrieval

4.1 상위 애플리케이션 흐름은 유지된다

src/graphs/main_graph.py의 StateGraph, MemorySaver, query rewrite, search scope, router, RDB path, VectorDB path, stock-price tool, final response 구조는 계속 앱의 질문 처리 흐름을 담당한다. GUI와 CLI도 graph_app.invoke를 호출한다. 따라서 langgraph와 langgraph-prebuilt는 V2 저장 구조와 별개의 상위 오케스트레이션 의존성이다.

[Streamlit GUI / CLI]

```
-> [LangGraph: rewrite, scope, route]
-> [src/nodes/vectoradb.py]
-> [bootstrap + dispatch]

    -> legacy_v1: 기존 LangChain FAISS
    -> epoch_zero_compatibility: sealed V1 bundle
    -> native: V2 SQLite catalog + raw FAISS

-> [기존 rerank / 답변 생성 / citation 흐름]
```

변경의 접점은 `src/nodes/vectordb.py`다. 이 노드는 기존 LangGraph State와 Document 기반 후속 처리를 유지하면서, 검색 backend를 `resolve_retrieval_dispatch`로 선택한다.

NativeRetrievalReader가 반환한 RetrievedChunk는 기존 후속 로직이 이해하는 Document 형태로 변환된다. 그 결과 상위 노드와 UI는 저장 구조 변경을 직접 알 필요가 없다.

4.2 v1과 v2의 역할 비교

관점	v1	v2
앱 오케스트레이션	LangGraph 노드 구조	동일한 LangGraph 노드 구조 사용
벡터 저장	LangChain FAISS index.faiss	Raw FAISS IndexIDMap2 snapshot
문서 매핑	index.pkl docstore와 ordinal mapping	SQLite snapshot_membership과 chunk metadata
활성 상태	파일 존재와 로딩 성공에 의존	retrieval_runtime singleton ⁰ authoritative pointer
갱신 방식	기존 index에 추가 저장	전체 corpus immutable successor publication
복구	파일 조합을 운영자가 판단	journal-checkpoint-committed floor 기반 자동 reconciliation
수명주기	지원 종료 전까지 보존	마이그레이션 후에도 계속 사용하는 영구 엔진

5. 소스 코드 구조와 수명주기 경계

소스 트리는 코드가 언제까지 필요한지에 따라 분리했다. src/retrieval은 v2가 활성화된 뒤 앱이 계속 사용하는 영구 패키지이고, src/migrations/v2와 scripts/migrations/v2는 v1에서 넘어오는 기간에만 필요한 전환 패키지다.

```
src/
  retrieval/          # 영구 v2 검색 엔진
  migrations/v2/      # V1 -> V2 전환 전용

scripts/migrations/v2/  # 운영자·사용자용 전환 진입점
docs/migrations/v2/    # 전환 기간 문서
MIGRATE_V2.bat         # 일반 사용자 원클릭 실행 파일
```

5.1 영구 retrieval 패키지

그룹	모듈	책임
진입·선택	bootstrap, dispatch	런타임 검사, v1/v2 모드 선택, process-scoped native reader 보유
조회	reader, repository, vector_index	검색 전략, snapshot lease, SQLite hydration, raw FAISS I/O
계약	schema, identity, manifest	카탈로그 제약, 결정적 UID, 전체 corpus completeness
쓰기	build_service, publication	전체 corpus 후보 생성, immutable snapshot 게시, active pointer 전환
복구	recovery, garbage_collector	시작 reconciliation, predecessor 복구, 안전한 retired artifact 정리
동시성	writer_lock, update_lock, runtime_guard	단일 writer 증명, update fence, 쓰기 가능 상태 확인
운영	launcher_guard, support_export	앱 실행 전 guard, 비밀·본문 없는 지원 상태 내보내기
임시 bridge	compatibility_bundle	epoch-zero sealed V1 bundle의 런타임 검증 계약

compatibility_bundle.py는 src/retrieval 안에 있지만 영구적인 v2 기능 자체가 아니라 전환 기간의 런타임 계약이다. 영구 패키지가 변환 로직을 import하지 않도록 bundle 생성은 src/migrations/v2/evidence.py에 두고, runtime 쪽에는 이미 봉인된 bundle을 검증하는 최소 계약만 남겼다.

5.2 전환 패키지

모듈	역할
assess	복사된 V1 DB-FAISS-pickle의 hash, count, dimension, metric, mapping 일관성을 읽기 전용으로 평가
evidence	SQLite online backup과 바이트 복사를 만들고 epoch-zero compatibility bundle을 읽기 전용으로 봉인
legacy_import	V1 docstore와 parent_chunks를 전체 N 기준으로 재구성
reconstruct	V1 embedding prefix를 제거하고 child body가 parent의 어느 span인지 유일하게 증명
import_v1	결정적 UID와 membership을 생성해 epoch-zero native seed로 변환
compatibility	epoch zero에서만 허용되는 sealed V1 reader
migrate_v2_user	표준 설치를 위한 원클릭 orchestration, cutover journal, 자동 복귀
migrate_v2_native	세부 단계를 직접 제어해야 하는 운영자용 진입점

의존성 방향 원칙: src/migrations/v2는 src/retrieval의 영구 계약을 사용할 수 있지만, src/retrieval은 src/migrations/v2를 사용하지 않는다. 예외적인 V1 reader 호출은 앱 통합 계층인 src/nodes/vectoradb.py의 epoch-zero branch에서만 lazy import한다.

6. 런타임 데이터 배치 구조

성공한 마이그레이션은 기존 data 디렉터리의 V1 파일을 교체하지 않고 data/retrieval만 추가한다. V2의 활성 카탈로그와 snapshot, publication 증거는 모두 이 하위 트리에 모인다. 마이그레이션 실행별 백업과 cutover 기록은 data와 같은 부모 아래 .v2m에 둔다.

```

project-root/
data/
  reports.db                # V1 호환 anchor, 보존
  vector_db/index.faiss     # V1 원본, 보존
  vector_db/index.pkl       # V1 원본, 보존
  downloaded/*.pdf          # 전체 source corpus
  conversations.db          # 검색 마이그레이션 대상 아님
  .retrieval-update.guard   # V1/V2 update 공통 fence
retrieval/
  migration-owner.json      # 원클릭 cutover 소유 표식
  compat/v1/<bundle_id>/     # epoch-zero V1 bridge
  v2/
    catalog.sqlite3         # V2 control plane과 logical catalog
    writer.guard            # persistent OS lock anchor
    writer.lock             # 실행 중 owner record
    snapshots/<snapshot_id>.faiss # immutable raw vector snapshot
    staging/                # off-path 임시 작성 위치
    evidence/<publication_id>/ # manifest, intent, committed floor
    backups/                # rollback copy와 committed checkpoint
  .v2m/<run_id>/            # 실행별 V1 backup-journal-receipt

```

6.1 Git과 런타임 상태의 분리

src/retrieval, src/migrations/v2, scripts/migrations/v2, docs/migrations/v2와 MIGRATE_V2.bat은 Git으로 관리하는 소스다. 반면 .v2m, data/retrieval/v2의 catalog-snapshot-evidence-lock, V1 backup은 설치별 런타임 데이터이므로 Git에 올리지 않는다. .gitignore의 /.v2m/ 규칙은 실행 증거와 백업이 소스 변경에 섞이지 않도록 한다.

V2 snapshot과 evidence는 다시 생성 가능한 단순 cache만은 아니다. 어떤 snapshot이 언제 게시되었는지와 복구 가능한 내구성 바닥을 나타내므로, 운영 중에는 애플리케이션이 정한 절차 외에 수동 삭제하거나 파일명을 바꾸면 안 된다. Git에서 제외하는 것과 운영상 중요하지 않다는 것은 다른 의미다.

7. V2 식별 체계와 카탈로그 데이터 모델

7.1 Embedding profile

V2의 논리 ID는 단순 파일명이나 배열 순서가 아니라 embedding 의미 공간을 포함한다.

EmbeddingProfile은 model, dimension, metric, normalization, prefix_template, extractor, parent_policy, child_policy를 하나의 불변 입력으로 묶고 canonical JSON과 SHA-256으로 profile_hash를 만든다. 모델이나 chunk 정책이 바뀌면 같은 PDF라도 다른 검색 의미 공간으로 취급된다.

7.2 결정적 논리 ID

```
report_uid = H(canonical path, source PDF hash, retrieval metadata hash)
parent_uid = H(profile hash, report_uid, parent order, parent content hash)
chunk_uid = H(profile hash, parent_uid, child order, span, embedding text hash)
faiss_id = chunk_uid를 byte 순으로 정렬한 snapshot-local 1..N
```

이 체계는 logical identity와 physical identity를 분리한다. report_uid·parent_uid·chunk_uid는 입력 의미가 같으면 snapshot이 바뀌어도 안정적이다. faiss_id는 해당 snapshot 안에서만 의미가 있는 조밀한 양의 정수다. 검색 결과를 hydrate할 때는 반드시 결과를 만든 snapshot revision과 동일한 membership을 사용해야 한다.

7.3 SQLite 카탈로그

테이블	책임
reports	source path, PDF hash, retrieval metadata hash, report type/date/target/title/broker와 report_uid 저장
embedding_profiles	모델·dimension·metric·normalization·추출기·chunk 정책과 profile_hash 저장
retrieval_parents	불변 parent 본문, 순서, content hash, report/profile 연결
retrieval_chunks	parent span, child order, embedding text hash와 chunk_uid 저장
retrieval_builds	전체 corpus manifest, included/excluded count, build 상태 저장
vector_snapshots	FAISS 경로, hash, size, dimension, metric, ntotal, snapshot 상태 저장
snapshot_membership	snapshot_id + chunk_uid + faiss_id의 정확한 대응 저장

테이블	책임
retrieval_runtime	active/predecessor pointer, publication generation, write epoch, fallback-degraded-writable 상태 저장
publication_runs	publication의 from/to snapshot, 단계, 상태, evidence hash와 오류 코드 저장

스키마는 외래키와 CHECK 제약뿐 아니라 상태 전이 trigger를 사용한다. 준비되지 않은 build를 active로 가리키거나, write_epoch가 증가한 뒤 V1 fallback을 다시 열거나, 완료된 publication을 실패 상태로 되돌리거나, immutable audit record를 삭제하는 모순을 데이터베이스 수준에서 막는다.

7.4 FAISS snapshot의 역할

V2 FAISS 파일은 IndexIDMap2로 구성되고 벡터와 1..N physical ID만 소유한다. 텍스트, Document 객체, metadata, docstore pickle은 포함하지 않는다. snapshot을 열 때 파일 hash, size, dimension, metric, ntotal, ID 조밀성, 벡터 유한값을 확인한다. Windows의 한글·Unicode 경로도 처리할 수 있도록 Python callback I/O를 사용한다.

7.5 전체 corpus manifest

발견된 모든 report_uid는 included 또는 허용된 excluded reason 중 하나로 정확히 한 번 결정되어야 한다. included report는 snapshot에 최소 한 개 chunk가 있어야 하고, excluded report에는 membership이 없어야 한다. 따라서 일부 PDF가 조용히 누락된 index는 ready snapshot이 될 수 없다.

8. v1에서 v2로 전환되는 상세 과정

일반 사용자는 MIGRATE_V2.bat을 실행한다. 이 파일은 scripts/migrations/v2/migrate_v2_user.py를 호출하며, 표준 data 배치와 parent-child V1을 대상으로 전체 흐름을 한 번에 수행한다. 이미 정상적인 writable V2가 있으면 새 backup이나 변환을 만들지 않고 현재 상태를 반환한다.

8.1 실행 전 조건

- data/reports.db, data/vector_db/index.faiss, data/vector_db/index.pkl이 존재하고 서로 같은 v1 상태여야 한다.
- reports.db에 기록된 모든 source PDF가 data/downloaded에 있어야 한다.
- 현재 설정의 embedding provider가 사용 가능하고 v1 벡터와 같은 의미 공간을 재현해야 한다.
- 원클릭 경로는 USE_PARENT_CHILD=true인 표준 v1 배치를 대상으로 한다.
- 쓰기 가능한 첫 successor에는 v1 seed에 없던 새 logical report가 최소 하나 포함되어야 한다.
- 앱, crawler, embedding updater와 마이그레이션이 동시에 동일 data root를 변경해서는 안 된다.

8.2 원클릭 전환의 8단계

1. v1 원본의 안전한 복사본 생성: reports.db는 SQLite online backup으로 일관된 복사본을 만들고, index.faiss와 index.pkl은 바이트 복사한다. 복사 전후 원본 hash가 달라지면 중단한다.
2. 원본 PDF와 successor 입력 확인: DB에 기록된 리포트와 source PDF의 SHA-256을 대응시키고, 전체 PDF inventory와 새 logical report 존재 여부를 확인한다. 필요하다면 제한된 최근 기간에서 새 리포트를 한 번 수집한다.
3. 격리된 위치에서 epoch-zero seed 변환: v1 bundle을 봉인하고, v1 parent/child 구조와 기존 벡터를 v2 UID-catalog-membership-raw FAISS snapshot으로 변환한다. 이 단계는 라이프 data/retrieval을 변경하지 않는다.
4. 동일 embedding 공간 확인: 현재 embedding provider가 기존 v1 벡터와 동일한 차원·방향·자기 검색 성질을 재현하는지 canary로 확인한다. 증명되지 않으면 v1을 유지한다.

5. 전체 corpus successor 생성: source PDF 전체를 다시 추출하고 chunk한 뒤 embedding 하여, 새 리포트를 포함한 쓰기 가능한 v2 candidate를 만든다. Candidate는 격리된 data root 안에서 완전히 materialize되고 publication된다.
6. 격리 v2 실행 확인: staged root가 예상 snapshot, publication generation, write epoch, fallback 상태를 가지는지 확인하고 실제 앱 진입점이 그 상태를 열 수 있는지 점검한다.
7. 최종 v1 상태 재확인: update fence 획득: .retrieval-update.guard를 잠근 뒤 v1 DB의 logical hash, FAISS/pickle hash, 전체 PDF membership과 bytes가 처음과 같은지 다시 확인한다.
8. 단일 rename으로 활성화: 완성된 staged retrieval 디렉터리에 owner marker와 cutover journal을 기록하고 data/retrieval로 rename한다. 라이브 상태를 다시 열어 staged identity와 정확히 같음을 확인한 뒤 receipt와 VERIFIED 상태를 기록한다.

전환의 핵심 안전성: 라이브 v1 파일을 v2 파일로 덮어쓰지 않는다. 활성화되는 것은 검증된 retrieval 디렉터리 하나이며, 활성화 실패 시 도구가 소유권과 native identity를 정확히 증명할 수 있는 경우에만 해당 디렉터를 격리하고 v1 선택 상태로 복귀한다.

8.3 재실행과 중단 복구

.v2m/<run_id>/cutover-journal.json은 PREPARED, ACTIVATED, VERIFIED, ROLLED_BACK 또는 MANUAL_SUPPORT 단계를 기록한다. 프로세스가 중단되면 다음 실행은 새 전환을 시작하기 전에 미완료 journal을 먼저 해석한다. 예상 snapshot, build, predecessor, publication generation, write epoch, owner marker가 모두 같을 때만 완료를 이어간다.

활성화 직후 문제가 생겼더라도 다른 writer가 더 새 snapshot을 게시한 흔적이 있으면 자동 rollback하지 않는다. 이 경우 기존 데이터를 임의로 되돌리는 것보다 MANUAL_SUPPORT로 중단하는 것이 안전하다. 성공 receipt를 기록한 뒤 rollback된 실행은 rolled-back-receipt.json으로 격리하여 더 이상 성공 증거로 오해하지 않게 한다.

9. 런타임 선택과 읽기 경로

9.1 런타임 상태

상태	선택 조건	읽기	쓰기
legacy_v1	V2 catalog과 durable footprint가 모두 없음	기존 V1 FAISS	V1 경로
native epoch 0	변환 seed가 정상, write_epoch=0, fallback open	V2 native seed	차단
epoch_zero_compatibility	epoch-zero active snapshot이 손상되고 sealed bundle이 유효	V1 sealed bundle	차단
native successor	write_epoch>0, fallback closed, active snapshot 정상	V2 native	허용
degraded native	active snapshot 문제, predecessor가 검증 가능	V2 predecessor	forward recovery 만
fail closed	catalog-evidence-snapshot 정체성을 증명할 수 없음	제공하지 않음	차단

V2 footprint가 전혀 없는 설치만 순수 V1로 판단한다. retrieval/v2나 compatibility 증거가 있는데 catalog이 사라졌다면 손상된 V2일 수 있으므로, 이를 fresh V1으로 오인해 legacy path를 다시 여는 대신 fail closed한다.

9.2 Native 요청 처리 순서

1. vectordb_node가 현재 질문과 metadata filter를 준비하고 resolve_retrieval_dispatch를 호출한다.
2. dispatch는 data root별 native holder가 이미 있으면 재사용하고, 없으면 bootstrap으로 catalog와 active snapshot을 확인한다.
3. 질문을 현재 embedding model로 query vector로 변환한다.
4. NativeRetrievalReader가 검색 scope를 SQL predicate와 bind parameter로 compile한다.

5. 하나의 SQLite read transaction과 하나의 active snapshot lease를 요청 전체에 고정한다.
6. eligible count와 전체 snapshot 크기에 따라 EMPTY, DIRECT, SELECTOR, ADAPTIVE 전략을 선택한다.
7. FAISS가 physical ID와 score를 반환한다. 결과에는 snapshot_id와 publication_generation이 함께 묶인다.
8. 동일 transaction의 snapshot_membership을 통해 chunk UID, parent slice, report metadata를 bulk hydrate한다.
9. RetrievedChunk를 기존 LangChain Document 형태로 변환해 기존 filter, coverage, rerank, answer 단계로 넘긴다.
10. 요청이 끝나면 lease를 해제한다. active pointer가 중간에 바뀌어도 해당 요청은 시작 시점의 revision만 사용한다.

9.3 검색 전략

전략	사용 조건	동작
EMPTY	명시적 empty scope, eligible=0 또는 snapshot N=0	FAISS를 호출하지 않고 빈 결과 반환
DIRECT	filter가 없거나 eligible count가 전체 N과 같음	FAISS에서 k개를 바로 검색하고 hydrate
SELECTOR	eligible 집합이 충분히 작고 전체 대비 비율도 낮음	SQLite에서 allowed physical IDs를 만든 뒤 FAISS selector 검색
ADAPTIVE	filter 집합이 넓어 selector보다 direct 후보 확장이 유리	작은 fetch_k부터 시작해 eligible 결과가 k에 도달할 때까지 점진 확대

가장 중요한 무결성 규칙은 cross-snapshot hydration 금지다. Physical ID 17은 snapshot A와 B에서 다른 chunk를 의미할 수 있으므로, 결과를 만든 revision과 hydrate transaction이 다르면 오류로 처리한다. Snapshot lease는 이 오류를 구조적으로 방지한다.

10. V2 쓰기와 전체 corpus successor 생성

V2가 활성화된 뒤 `src/core/embed_pipeline.py`는 더 이상 V1 index에 개별 document를 append 하지 않는다. Runtime guard가 native writable 상태를 확인하면 `execute_full_corpus_successor`를 호출한다. `--limit` 같은 부분 처리 값은 V2 publication 의미를 만들지 못하므로 전체 corpus를 기준으로 처리한다.

10.1 쓰기 경로

1. RetrievalUpdateLock으로 `crawler.embedding.migration` 간 data root 변경을 직렬화한다.
2. NativeWriterLock으로 candidate 계획부터 publication 종료까지 하나의 writer lease를 유지한다.
3. StartupReconciler가 이전 실행의 미완료 publication이나 pending garbage를 먼저 정리한다.
4. source directory 전체를 inventory하고 각 PDF hash와 metadata를 읽어 deterministic build plan을 만든다.
5. PDF를 추출하고 parent/child chunk를 만들며 embedding profile에 맞는 prefix와 vector를 생성한다.
6. `report.parent.chunk UID`와 `1..N physical ID`를 계산하고 corpus manifest를 확정한다.
7. staging 위치에 raw FAISS를 만들고 다시 열어 descriptor와 vector payload를 확인한다.
8. catalog에 build, snapshot, membership을 삽입하고 ready 상태까지 전이한다.
9. candidate evidence를 read-only JSON으로 기록한 뒤 PublicationCoordinator에 전달한다.
10. publication이 완료되면 새 snapshot이 active가 되고 직전 active는 predecessor가 된다.

10.2 전체 corpus 방식의 이유

부분 append는 active snapshot 파일과 mapping을 직접 변경하게 만들고, 삭제·교체된 PDF가 index에 남는 문제를 만든다. v2는 매번 발견된 source 전체를 included/excluded로 결정하고, 완전한 successor를 off-path에서 만든다. Active snapshot은 publication 전까지 바뀌지 않으므로 읽기 요청은 안정된 revision을 계속 사용할 수 있다.

동일 입력에서는 build_id, snapshot_id, publication_id와 vector payload hash가 결정적으로 연결된다. 중단 후 동일 candidate가 이미 존재하면 내용이 정확히 같은 경우에만 채택하고, 같은 identity 경로에 다른 bytes가 있으면 충돌로 중단한다.

10.3 추출·chunk 정책의 연속성

첫 successor는 저장 구조는 v2로 바뀌되 검색 의미를 불필요하게 바꾸지 않는다. v1의 parent/child 크기, overlap, Markdown header 분할, embedding prefix, extraction fallback 정책을 profile에 명시한다. 설정된 비-PyMuPDF 추출기가 실패하거나 빈 결과를 내면 허용된 경우 PyMuPDF fallback을 사용하되, 이 정책 자체를 extractor fingerprint에 포함한다.

11. 원자적 publication과 내구성 프로토콜

Publication의 목적은 파일을 복사하는 것이 아니라 새 snapshot을 active로 선언하는 단일 논리적 사건을 내구성 있게 기록하는 것이다. SQLite retrieval_runtime이 유일한 active pointer 권위이며, 파일명이나 가장 최근 수정 시각을 보고 active snapshot을 선택하지 않는다.

11.1 주요 publication 단계

구간	내구성 기록	의미
Journal	publication_runs 행	from/to snapshot과 evidence identity를 고정하고 재실행 입력을 불변화
Candidate	artifact hash-descriptor 확인	ready snapshot이 실제 파일 bytes와 일치함을 확인
Rollback floor	catalog rollback copy	pointer commit 전 상태를 복구할 수 있는 기반 확보
Commit intent	evidence/<id>/commit-intent.json	목표 generation-epoch-fallback floor-target snapshot을 transaction 전에 내구화
Pointer commit	retrieval_runtime transaction	active와 predecessor를 원자적으로 교체하고 generation-epoch 증가
Checkpoint	catalog-current-g*.sqlite3	commit 후 카탈로그 전체를 독립적으로 복구 가능한 형태로 고정
Committed floor	committed-floor.json	되돌아갈 수 없는 최소 generation-write epoch-fallback 상태 선언
Complete	publication fully_complete	build를 fully_complete로 전이하고 retired artifact 정리 시작

11.2 publication generation과 write epoch

publication_generation은 active pointer 변경의 순서를 나타내며 뒤로 갈 수 없다. write_epoch는 쓰기 가능한 native successor의 세대를 나타낸다. 첫 v1 변환 seed는 write_epoch=0이고

fallback floor가 open이다. 첫 successor가 게시되면 write_epoch가 양수가 되고 fallback floor가 closed로 내려가며, 이후 다시 open으로 올라갈 수 없다.

predecessor_snapshot_id는 직전 active snapshot을 가리킨다. Active snapshot을 열 수 없을 때 verified predecessor를 사용해 degraded read-only 상태로 서비스할 수 있지만, 이를 정상 active로 가장하지 않는다. 다음 full-corpus build를 통한 forward recovery가 성공해야 정상 writable 상태로 복귀한다.

11.3 Immutable artifact 원칙

- Snapshot, candidate evidence, commit intent, committed floor는 고유 이름으로 한 번만 게시한다.
- 기존 경로를 replace하거나 overwrite해 다른 bytes를 같은 identity로 보이게 하지 않는다.
- Snapshot 경로는 catalog 상대 경로로만 저장하고 data root 밖으로 나가는 absolute path·traversal·symlink를 거부한다.
- 완료된 publication과 vector snapshot 행은 audit record이므로 임의 삭제나 상태 후퇴를 허용하지 않는다.
- Commit 후에는 checkpoint와 committed floor가 검증되기 전까지 publication을 완전히 끝난 것으로 간주하지 않는다.

12. 동시성 제어와 잠금 계층

V2는 하나의 lock만으로 모든 문제를 해결하지 않는다. Cutover 중 V1 source 변경을 막는 fence와, native build/publication의 단일 writer를 증명하는 lease가 서로 다른 위치와 책임을 가진다.

장치	위치	보호 범위
RetrievalUpdateLock	data/.retrieval-update.guard	crawler, embedding updater, 원클릭 migration 간 source/data-root 변경 직렬화

장치	위치	보호 범위
writer.guard	data/retrieval/v2/writer.guard	OS가 소유하는 persistent native writer lock anchor
writer.lock	data/retrieval/v2/writer.lock	hostname, PID, process identity, nonce, 생성 시각을 담는 transient owner record
WriterLease	프로세스 메모리 + guard descriptor	candidate 계획부터 publication과 정리까지 같은 writer가 계속 소유했음을 증명
Runtime guard	쓰기 진입점에서 catalog 검사	epoch zero, fallback, degraded, running publication, pending garbage 상태에서 일반 쓰기 차단

writer.lock은 먼저 임시 파일에 완전한 JSON을 쓰고 file sync한 뒤 hard link로 authoritative 이름을 게시한다. Partial owner record가 보이지 않도록 하고, 기존 owner record를 overwrite하지 않는다. Stale lock 회수도 단순 PID 확인이 아니라 hostname, process creation identity, nonce, 나이를 함께 확인한 뒤 별도 quarantine 이름으로 이동한다.

운영자가 writer.guard나 writer.lock을 임의로 삭제해 blocked writer를 우회해서는 안 된다. 현재 소유권을 증명할 수 없는 상태에서 lock을 제거하면 두 writer가 같은 catalog와 snapshot tree를 동시에 변경할 수 있다.

13. 시작 복구, 정리 및 지원 정보

13.1 Startup reconciliation

앱 시작 시 reconcile_and_inspect_runtime은 native writer lock을 잡고 StartupReconciler를 먼저 실행한다. 복구기는 파일명 순서가 아니라 integrity-checked catalog, publication journal, commit intent, committed floor, checkpoint를 사용한다. 그 결과는 다음 중 정확히 하나다.

결과	설명
ACTIVE	현재 active snapshot과 catalog가 정상
PREVIOUS_ACTIVE	미완료 pre-commit publication을 정리하고 기존 active 유지
PUBLICATION_COMPLETED	내구성 증거와 pointer가 일치하는 미완료 publication을 끝까지 재생
CHECKPOINT_RESTORED	catalog 손상 시 가장 높은 검증된 committed floor가 지정한 checkpoint 복원
PREDECESSOR_DEGRADED	active가 손상됐지만 predecessor가 유효하여 read-only degraded 상태로 전환

결과	설명
V1_FALLBACK	epoch zero에서만 sealed compatibility bundle을 사용
FAIL_CLOSED	서로 모순된 증거이거나 안전한 복구 대상을 증명할 수 없음

13.2 Garbage collection

Retired snapshot은 즉시 삭제하지 않는다. Active나 predecessor가 아니고, 열려 있는 snapshot lease가 없고, durable recovery가 해당 파일을 더 이상 요구하지 않는다는 조건을 충족한 뒤 garbage_pending에서 garbage_collected로 전이한다. 삭제가 파일 잠금이나 권한 문제로 실패하면 pending 상태를 남기고 다음 clean startup에서 재시도한다.

Epoch-zero compatibility bundle도 첫 successor가 fallback을 닫았다는 사실만으로 바로 지우지 않는다. Publication은 cleanup-pending marker를 기록하고, 별도의 retention 승인과 validation window 종료가 명시된 뒤에만 intent와 complete evidence를 남기며 정리한다.

13.3 Support export

support_export는 운영 점검에 필요한 환경 버전, runtime mode, active/predecessor identity, generation-epoch, snapshot descriptor, catalog count, build/snapshot state count, compatibility 상태, committed floor와 publication 요약을 JSON으로 내보낸다. API key, query text, report 본문, vector, 절대 source path는 포함하지 않는다. 출력 자체도 manifest hash를 포함하고 기존 파일을 덮어쓰지 않는다.

14. 애플리케이션 통합 방식

14.1 앱 시작

RUN_APP.bat과 RUN_QUICKSTART.bat은 앱 프로세스를 시작하기 전에 launcher_guard를 통과한다. Guard는 native footprint가 있다면 startup reconciliation과 snapshot 검증을 수행하고, 선택된 runtime identity를 구조화된 형태로 확인한다. 쓰기 작업을 시작하는 경우에는 --write 의미로 writable 상태까지 요구한다.

14.2 검색 노드

src/nodes/vectordb.py는 세 backend를 한 함수에서 다룬다. 순수 V1에서는 LangChain FAISS를, epoch-zero fallback에서는 V1CompatibilityReader를, 정상 V2에서는 process-scoped NativeRetrievalReader를 사용한다. Native 결과는 기존 metadata filter, document coverage, optional rerank, citation, final response 흐름에 맞게 Document로 변환한다.

14.3 데이터 업데이트

src/core/data_update_jobs.py, report_crawler.py, embed_pipeline.py는 공통 runtime guard와 update lock을 사용한다. V1에서는 기존 수집·임베딩 흐름이 유지되고, 정상 V2에서는 crawler가 source corpus를 갱신한 뒤 embedding pipeline이 whole-corpus successor를 만든다. Epoch zero에서는 일반 updater가 쓰기를 시작할 수 없고, 첫 successor 전용 writer 흐름만 허용된다.

14.4 상태 표시

앱 상태 계층은 runtime mode, snapshot, publication generation, write epoch, fallback open 여부, degraded 여부, write enabled 여부를 기준으로 V2 상태를 표시할 수 있다. 사용자는 V2 내부 테이블을 직접 다룰 필요가 없지만, 운영자는 이 값들을 통해 단순 V1, epoch zero, 정상 successor, degraded 상태를 구분할 수 있다.

LangGraph는 삭제되지 않았다. V2 migration으로 제거된 것은 V2 hot path에서 LangChain FAISS docstore/pickle wrapper에 의존하던 저장 방식이다. 질문 처리 graph, message, prompt, tool node는 계속 유지된다.

15. V1 보존 정책과 지원 종료 이후 정리

15.1 현재 보존되는 V1

- 라이브 `data/reports.db`와 `data/vector_db`는 원클릭 전환이 성공해도 삭제하거나 교체하지 않는다.
- `.v2m/<run_id>/v1`에는 실행 시점의 `copied-install backup`과 `manifest`가 남는다.
- `data/retrieval/compat/v1/<bundle_id>`에는 `epoch-zero fallback`에 필요한 `read-only bundle`이 보존된다.
- `src/nodes/vectordb.py`의 `legacy_v1 branch`는 V2 footprint가 없는 기존 사용자를 계속 지원한다.
- `src/migrations/v2`는 아직 전환하지 않은 사용자의 V1 데이터를 V2로 옮기기 위해 유지한다.

15.2 V1 지원 종료 이후 삭제 가능한 범위

삭제 대상	전제 조건
<code>src/migrations/v2</code>	지원 대상 설치가 모두 V2로 전환되고 V1 import를 더 제공하지 않음
<code>scripts/migrations/v2</code>	운영자·사용자 전환 명령을 더 배포하지 않음
<code>MIGRATE_V2.bat</code>	원클릭 V1 전환 진입점 종료
<code>docs/migrations/v2</code>	전환 <code>runbook</code> 과 사용자 문서의 운영 수명 종료
<code>src/retrieval/compatibility_bundle.py</code>	<code>epoch-zero V1 bridge</code> 계약을 더 사용할 필요가 없음
<code>bootstrap</code> 의 <code>epoch-zero branch</code>	V1 fallback을 허용할 설치가 없음
<code>vectordb.py</code> 의 <code>legacy-compatibility branch</code>	앱이 <code>native V2</code> 만 지원
기존 V1 FAISS writer 코드	모든 데이터 갱신이 V2 whole-corpus publication만 사용

반대로 src/retrieval의 native bootstrap, dispatch, reader, repository, schema, vector_index, identity, manifest, build_service, publication, recovery, garbage_collector, writer_lock, update_lock, runtime_guard, launcher_guard, support_export는 V2 운영의 본체이므로 삭제 대상이 아니다.

15.3 의존성 정리 관점

V1 지원 종료 후 legacy FAISS reader와 writer, migration importer가 모두 제거되면 langchain-community는 제거 후보가 된다. 그러나 langgraph와 langgraph-prebuilt는 저장 구조와 무관하게 앱 오케스트레이션과 ToolNode에서 계속 사용되므로 V1 종료만으로 제거할 수 없다. langchain-core, langchain-text-splitters, langchain-openrouter 역시 현재 앱과 V2 build에서 직접 사용한다.

16. 운영 제약과 핵심 원칙

16.1 원클릭 경로의 제약

- 표준 data 디렉터리 배치를 전제로 하며 DB_PATH, FAISS_DIR, SAVE_DIR가 서로 다른 custom root인 설치는 자동 전환하지 않는다.
- 현재 자동 변환은 parent-child V1 계약을 전제로 한다. 단일 레벨 chunk V1은 별도 이관 설계가 필요하다.
- 모든 DB-visible report의 원본 PDF가 있어야 전체 corpus successor를 만들 수 있다.
- 현재 embedding provider가 V1 생성 당시와 같은 벡터 공간을 재현하지 못하면 기존 벡터 import와 first successor 사이의 연속성을 증명할 수 없다.
- 새 logical report가 없으면 first successor는 seed와 구별되는 쓰기 세대를 만들 수 없으므로 V1을 유지한 채 중단한다.
- Snapshot과 catalog는 같은 로컬 data root 안의 실제 파일이어야 하며 symlink, junction, path traversal을 허용하지 않는다.

16.2 유지보수자가 지켜야 할 원칙

1. Active snapshot을 파일명이나 최신 수정 시각으로 선택하지 말고 `retrieval_runtime`만 신뢰한다.
2. Physical ID를 snapshot 밖의 영구 ID로 사용하지 않는다. 영구 정체성은 `chunk_uid`다.
3. Active snapshot 파일을 in-place로 변경하지 않는다. 항상 새 immutable successor를 만든다.
4. V1 fallback은 epoch zero에서만 허용하고 `write_epoch`가 증가한 뒤 다시 열지 않는다.
5. 전환 코드의 편의를 위해 `src/retrieval`이 `src/migrations`를 import하도록 의존성을 뒤집지 않는다.
6. Lock 파일을 수동 제거해 안전장치를 우회하지 않는다. Ownership을 증명할 수 없으면 중단한다.
7. Publication evidence와 backup을 일반 cache처럼 정리하지 않는다. Garbage collector와 retention 조건을 따른다.
8. V1 지원 종료는 별도 결정으로 처리하고, transition tree와 runtime bridge를 한 번에 정리한다.

v2의 기본 철학은 '정상일 가능성이 높다'가 아니라 '어떤 bytes와 어떤 generation을 선택해야 하는지 증명할 수 있다'이다. 증명할 수 없는 상태는 자동 추측보다 fail closed가 우선한다.

17. 구현 파일 안내와 용어 정리

17.1 주요 구현 파일

파일	확인할 내용
src/retrieval/schema.py	V2 SQLite 테이블, trigger, 상태 전이와 불변성 제약
src/retrieval/identity.py	embedding profile과 report/parent/chunk UID 계산
src/retrieval/vector_index.py	raw FAISS snapshot 생성·검증·검색·파일 I/O
src/retrieval/repository.py	snapshot revision lease와 SQLite hydration
src/retrieval/reader.py	scope compile과 EMPTY/DIRECT/SELECTOR/ADAPTIVE 전략
src/retrieval/bootstrap.py	legacy/native/compatibility 선택과 startup validation
src/retrieval/dispatch.py	data root별 process-scoped native reader
src/retrieval/build_service.py	전체 corpus 계획·materialize·publish orchestration
src/retrieval/publication.py	journal, intent, pointer commit, checkpoint, committed floor
src/retrieval/recovery.py	중단 publication과 손상 snapshot의 startup reconciliation
src/retrieval/writer_lock.py	nonce와 process identity를 포함한 native single-writer lease
src/retrieval/garbage_collector.py	retired snapshot과 compatibility bundle 정리
src/migrations/v2/import_v1.py	V1 full-N 구조를 epoch-zero V2 seed로 변환
scripts/migrations/v2/migrate_v2_user.py	일반 사용자용 8단계 cutover와 restart recovery
src/nodes/vectordb.py	LangGraph VectorDB node와 V1/V2 backend 연결 경계

17.2 용어

용어	의미
Epoch zero	V1 벡터를 V2 catalog와 raw FAISS로 변환한 최초의 read-only native seed
Successor	현재 active snapshot 전체를 대체하는 완전한 다음 full-corpus snapshot
Snapshot	특정 build의 immutable FAISS 파일과 SQLite membership의 결합

용어	의미
Revision	publication_generation과 snapshot_id로 고정된 한 요청의 읽기 관점
Publication	ready candidate를 active로 원자적으로 전환하고 내구성 증거를 완성하는 프로토콜
Committed floor	복구가 그보다 과거 generation-epoch-fallback 상태로 내려갈 수 없음을 나타내는 증거
Compatibility bundle	epoch zero에서만 선택 가능한 hash-validated read-only V1 DB-FAISS-pickle 묶음
Degraded	검증된 predecessor로 읽기는 가능하지만 일반 쓰기는 막힌 상태
Fail closed	안전한 runtime identity를 증명할 수 없을 때 검색 또는 쓰기를 중단하는 정책
Full corpus	발견된 모든 source report를 included 또는 명시적 excluded로 결정한 완전한 입력 집합

17.3 최종 요약

Finance LLM V2 migration은 기존 RAG 애플리케이션을 다시 작성하지 않고 검색 저장 계층의 신뢰 모델을 바꿨다. V1의 FAISS + pickle 결합을 SQLite logical catalog + immutable raw FAISS snapshot으로 분리하고, 결정적 identity, whole-corpus build, single-writer lease, crash-safe publication, startup recovery를 추가했다.

전환 기간에는 V1 원본과 sealed compatibility bundle을 보존하고, 첫 writable successor가 publication된 뒤 V1 fallback을 영구적으로 닫는다. 소스 코드도 영구 retrieval과 transition migration으로 분리했기 때문에, 향후 V1 지원 종료 시 무엇을 제거하고 무엇을 남겨야 하는지 폴더와 의존성 방향만으로 판단할 수 있다.

최종 상태: LangGraph는 질문 처리 오케스트레이션으로 유지되고, V2 Retrieval은 SQLite가 의미와 active pointer를 소유하며 FAISS는 불변 벡터 snapshot만 소유한다.